week_5_ledger_analysis_day_1.py

```
# -*- coding: utf-8 -*-
"""Week 5 Ledger Analysis day 1.ipynb

Automatically generated by Colab.

Original file is located at
    https://colab.research.google.com/drive/1ZAUkLeMoMr-NjzG5EsKCmjOyJuBJMX0L

# Week 5 Ledger Analysis day 1
"""

import pandas as pd

ledger = pd.read_csv("/content/stock_ledger.csv", parse_dates=["movement_date"])
inv = pd.read_csv("/content/inventory_master.csv")

print("Movement types:")
print(ledger["movement_type"].value_counts().to_string())
print("\nReference types:")
print(ledger["reference_type"].value_counts().to_string())
print("\nDate range:", ledger["movement_date"].min().date(), "to",
    ledger["movement_date"].max().date())

# Latest running balance for each product and branch
last = (ledger.sort_values(["movement_date", "movement_id"])
        .groupby(["product_id", "branch_id"])
        .tail(1)[["product_id", "branch_id", "running_balance"]])

check = inv.merge(last, on=["product_id", "branch_id"], how="outer", indicator=True)
print("\nMatch status:")
print(check["_merge"].value_counts().to_string())

check["diff"] = check["current_stock"] - check["running_balance"]
print("\nRows where current_stock equals ledger balance:", int((check["diff"] == 0).sum()), "of",
    len(check))
print("Total current_stock:", f"{check['current_stock'].sum():,.0f}")
print("Total ledger balance:", f"{check['running_balance'].sum():,.0f}")
print("\nSample mismatches:")
print(check.loc[check["diff"] != 0, ["product_id", "branch_id", "current_stock", "running_balance",
    "diff"]].head(10).to_string(index=False))

"""# Week 5 Ledger Analysis day 2"""

import pandas as pd
from pathlib import Path

OUT = Path("/content/week-05/output")
OUT.mkdir(parents=True, exist_ok=True)
DAYS = 2192  # 2019-01-01 to 2024-12-31

ledger = pd.read_csv("/content/stock_ledger.csv", parse_dates=["movement_date"])
products = pd.read_csv("/content/products.csv")
perf = pd.read_csv("/content/product_performance.csv")

ledger["month"] = ledger["movement_date"].dt.to_period("M")

# Last balance in each month for each product and branch
month_end = (ledger.sort_values(["movement_date", "movement_id"])
             .groupby(["product_id", "branch_id", "month"])
             .tail(1)[["product_id", "branch_id", "month", "running_balance"]])

# Full grid of every product-branch-month, carrying balances forward
months = pd.period_range("2019-01", "2024-12", freq="M")
pairs = ledger[["product_id", "branch_id"]].drop_duplicates()
grid = pairs.merge(pd.DataFrame({"month": months}), how="cross")
grid = grid.merge(month_end, on=["product_id", "branch_id", "month"], how="left")
grid = grid.sort_values(["product_id", "branch_id", "month"])
grid["running_balance"] = (grid.groupby(["product_id", "branch_id"])["running_balance"]
                           .ffill().fillna(0))
```

```
print("Negative month-end balances:", int((grid["running_balance"] < 0).sum()))

# Value at cost
grid = grid.merge(products[["product_id", "unit_cost"]], on="product_id", how="left")
grid["value"] = grid["running_balance"] * grid["unit_cost"]

monthly_value = grid.groupby("month")["value"].sum()
avg_inventory_value = monthly_value.mean()

# Turnover on average inventory
cogs_total = perf["cogs"].sum()
turnover = cogs_total / avg_inventory_value
print(f"Total COGS: {cogs_total:,.0f}")
print(f"Average inventory value (month-end): {avg_inventory_value:,.0f}")
print(f"Turnover: {turnover:.2f}x")
print(f"Days of inventory: {DAYS / turnover:,.0f}")

# Turnover by product
avg_by_product = (grid.groupby("product_id")["value"].mean()
                  .rename("avg_inventory_value").reset_index())
prod = perf[["product_id", "product_name", "cogs"]].merge(avg_by_product, on="product_id",
    how="left")
prod["turnover"] = prod["cogs"] / prod["avg_inventory_value"]
prod["days_of_inventory"] = DAYS / prod["turnover"]
prod.sort_values("turnover").to_csv(OUT / "week5_turnover_by_product.csv", index=False)
print("\nSaved:", OUT / "week5_turnover_by_product.csv")
print(prod.sort_values("turnover").head(5).to_string(index=False))

"""# Week 5 Ledger Analysis day 3

---


"""

import pandas as pd
from collections import deque
from pathlib import Path

OUT = Path("/content/week-05/output")
OUT.mkdir(parents=True, exist_ok=True)

ledger = pd.read_csv("/content/stock_ledger.csv", parse_dates=["movement_date"])
ledger = ledger.sort_values(["movement_date", "movement_id"]).reset_index(drop=True)
AS_OF = ledger["movement_date"].max()

# Per-row change in balance, derived from running_balance
ledger["prev_bal"] = ledger.groupby(["product_id",
    "branch_id"])["running_balance"].shift(1).fillna(0)
ledger["delta"] = ledger["running_balance"] - ledger["prev_bal"]

# Sign check: does the balance change match the recorded quantity?
print("Sign check by movement type:")
for mt, g in ledger.groupby("movement_type"):
    print(f"{mt:12s} rows={len(g):>7,}  delta==qty: {(g['delta'] == g['quantity']).mean():.3f}"
          f"  delta==-qty: {(g['delta'] == -g['quantity']).mean():.3f}")

# FIFO: build lots of stock still on hand, oldest sold first
lots_out = []
for (pid, bid), g in ledger.groupby(["product_id", "branch_id"]):
    q = deque()  # each item: [receipt_date, remaining_qty]
    for d, delta, date in zip(g["delta"], g["delta"], g["movement_date"]):
        if delta > 0:
            q.append([date, delta])
        elif delta < 0:
            need = -delta
            while need > 0 and q:
                take = min(need, q[0][1])
                q[0][1] -= take
                need -= take
```

```
                if q[0][1] == 0:
                    q.popleft()
    for date, rem in q:
        lots_out.append((pid, bid, date, rem))

lots = pd.DataFrame(lots_out, columns=["product_id", "branch_id", "lot_date", "qty"])
print("\nUnits on hand from FIFO lots:", f"{lots['qty'].sum():,.0f}",
      "(should match total current_stock, 19,303,266)")

lots["age_days"] = (AS_OF - lots["lot_date"]).dt.days
lots["age_bucket"] = pd.cut(lots["age_days"], bins=[-1, 30, 90, 180, 365, 10**6],
                            labels=["0-30", "31-90", "91-180", "181-365", ">365"])

products = pd.read_csv("/content/products.csv")[["product_id", "product_name", "unit_cost"]]
lots = lots.merge(products, on="product_id", how="left")
lots["value"] = lots["qty"] * lots["unit_cost"]

aging = lots.groupby("age_bucket", observed=False).agg(units=("qty", "sum"), value=("value", "sum"))
aging["pct_of_value"] = (aging["value"] / aging["value"].sum() * 100).round(1)
print(f"\nStock on hand by age, as of {AS_OF.date()}:")
print(aging.to_string())
aging.to_csv(OUT / "week5_aging_buckets.csv")

per_prod = (lots.assign(w=lots["age_days"] * lots["qty"])
            .groupby(["product_id", "product_name"])
            .agg(units=("qty", "sum"), value=("value", "sum"),
                 w=("w", "sum"), oldest_days=("age_days", "max"))
            .reset_index())
per_prod["avg_age_days"] = (per_prod["w"] / per_prod["units"]).round(0)
per_prod.drop(columns="w").sort_values("avg_age_days", ascending=False).to_csv(
    OUT / "week5_aging_by_product.csv", index=False)
print("\nOldest-on-average products:")
print(per_prod.sort_values("avg_age_days", ascending=False).head(5)[
    ["product_name", "avg_age_days", "oldest_days", "value"]].to_string(index=False))

import pandas as pd

ledger = pd.read_csv("/content/stock_ledger.csv", parse_dates=["movement_date"])

print("Sample movement_ids:", ledger["movement_id"].head(5).tolist())
print("Sorted by id, first 5:", ledger["movement_id"].sort_values().head(5).tolist())

print("\nTotal quantity by type:")
print(ledger.groupby("movement_type")["quantity"].agg(["sum", "min", "max"]).to_string())

# Does the balance chain fit better sorted by id than by date?
for key in [["movement_date", "movement_id"], ["movement_id"]]:
    s = ledger.sort_values(key).copy()
    s["prev"] = s.groupby(["product_id", "branch_id"])["running_balance"].shift(1).fillna(0)
    s["delta"] = s["running_balance"] - s["prev"]
    ins = s[s["movement_type"] == "IN"]
    outs = s[s["movement_type"] == "OUT"]
    print(key, "| IN match:", round((ins["delta"] == ins["quantity"]).mean(), 3),
          "| OUT match:", round((outs["delta"] == -outs["quantity"]).mean(), 3))

import pandas as pd

ledger = pd.read_csv("/content/stock_ledger.csv", parse_dates=["movement_date"])
print("Sample dates:", ledger["movement_date"].head(3).tolist())

s = ledger.sort_values(["movement_date", "movement_id"]).copy()
s["prev"] = s.groupby(["product_id", "branch_id"])["running_balance"].shift(1).fillna(0)
s["delta"] = s["running_balance"] - s["prev"]
s["ok"] = (((s["movement_type"] == "IN") & (s["delta"] == s["quantity"])) |
           ((s["movement_type"] == "OUT") & (s["delta"] == -s["quantity"])) |
           (s["movement_type"] == "ADJUSTMENT"))

bad = s[~s["ok"]]
print("Mismatched IN/OUT rows:", len(bad))
```

```
per_day = s.groupby(["product_id", "branch_id", "movement_date"])["movement_id"].transform("size")
print("Share of mismatches on a day with several movements:",
      round((per_day[bad.index] > 1).mean(), 3))
print("Share of all rows on a day with several movements:",
      round((per_day > 1).mean(), 3))
print("\nMismatches by year:")
print(bad.groupby(bad["movement_date"].dt.year).size().to_string())
print("\nFirst mismatches:")
print(bad.head(5)[["movement_id", "product_id", "branch_id", "movement_type",
                   "movement_date", "quantity", "running_balance", "delta"]].to_string(index=False))

"""# Week 5 Ledger Analysis day 3

---




"""

import pandas as pd
from pathlib import Path

OUT = Path("/content/week-05/output")
DAYS = 2192

prod = pd.read_csv(OUT / "week5_turnover_by_product.csv")
products = pd.read_csv("/content/products.csv")[["product_id", "category"]]
prod = prod.merge(products, on="product_id", how="left")

cat = prod.groupby("category").agg(
    products=("product_id", "size"),
    cogs=("cogs", "sum"),
    avg_inventory=("avg_inventory_value", "sum"))
cat["turnover"] = cat["cogs"] / cat["avg_inventory"]
cat["days_of_inventory"] = DAYS / cat["turnover"]
cat = cat.sort_values("turnover")
cat.to_csv(OUT / "week5_turnover_by_category.csv")
print(cat.round(2).to_string())

# Sales header columns, needed for turnover by branch
print("\nSales header columns:",
      pd.read_csv("/content/sales_orders_header.csv", nrows=0).columns.tolist())

"""# Week 5 Ledger Analysis day 4

---


"""

import pandas as pd
from pathlib import Path

OUT = Path("/content/week-05/output")
OUT.mkdir(parents=True, exist_ok=True)
DAYS = 2192

products = pd.read_csv("/content/products.csv")[["product_id", "unit_cost"]]
headers = pd.read_csv("/content/sales_orders_header.csv", usecols=["so_id", "branch_id"])
lines = pd.read_csv("/content/sales_orders_lines.csv")

# COGS by branch: sales lines joined to their order header
lines = lines[lines["quantity"] > 0].merge(headers, on="so_id", how="left")
lines = lines.merge(products, on="product_id", how="left")
lines["cogs"] = lines["quantity"] * lines["unit_cost"]
cogs_branch = lines.groupby("branch_id")["cogs"].sum().rename("cogs")
print("Lines with no branch:", int(lines["branch_id"].isna().sum()))
```

```
# Average month-end inventory value by branch
ledger = pd.read_csv("/content/stock_ledger.csv", parse_dates=["movement_date"])
ledger["month"] = ledger["movement_date"].dt.to_period("M")
month_end = (ledger.sort_values(["movement_date", "movement_id"])
             .groupby(["product_id", "branch_id", "month"]).tail(1)
             [["product_id", "branch_id", "month", "running_balance"]])

months = pd.period_range("2019-01", "2024-12", freq="M")
pairs = ledger[["product_id", "branch_id"]].drop_duplicates()
grid = (pairs.merge(pd.DataFrame({"month": months}), how="cross")
        .merge(month_end, on=["product_id", "branch_id", "month"], how="left")
        .sort_values(["product_id", "branch_id", "month"]))
grid["running_balance"] = (grid.groupby(["product_id", "branch_id"])["running_balance"]
                           .ffill().fillna(0))
grid = grid.merge(products, on="product_id", how="left")
grid["value"] = grid["running_balance"] * grid["unit_cost"]

avg_branch = (grid.groupby(["branch_id", "month"])["value"].sum()
              .groupby("branch_id").mean().rename("avg_inventory"))

br = pd.concat([cogs_branch, avg_branch], axis=1)
br["turnover"] = br["cogs"] / br["avg_inventory"]
br["days_of_inventory"] = DAYS / br["turnover"]
br = br.sort_values("turnover")
br.to_csv(OUT / "week5_turnover_by_branch.csv")
print(br.round(2).to_string())

import pandas as pd
from pathlib import Path

OUT = Path("/content/week-05/output")
DAYS = 2192

ledger = pd.read_csv("/content/stock_ledger.csv", parse_dates=["movement_date"])
products = pd.read_csv("/content/products.csv")[["product_id", "unit_cost", "category"]]
ledger["month"] = ledger["movement_date"].dt.to_period("M")

month_end = (ledger.sort_values(["movement_date", "movement_id"])
             .groupby(["product_id", "branch_id", "month"]).tail(1)
             [["product_id", "branch_id", "month", "running_balance"]])
months = pd.period_range("2019-01", "2024-12", freq="M")
pairs = ledger[["product_id", "branch_id"]].drop_duplicates()
grid = (pairs.merge(pd.DataFrame({"month": months}), how="cross")
        .merge(month_end, on=["product_id", "branch_id", "month"], how="left")
        .sort_values(["product_id", "branch_id", "month"]))
grid["running_balance"] = (grid.groupby(["product_id", "branch_id"])["running_balance"]
                           .ffill().fillna(0))
grid = grid.merge(products, on="product_id", how="left")
grid["value"] = grid["running_balance"] * grid["unit_cost"]

# Total value per product per month across all branches, then averaged over months
prod_avg = (grid.groupby(["product_id", "month"])["value"].sum()
            .groupby("product_id").mean().rename("avg_inventory_value"))
print("Sum of product averages:", f"{prod_avg.sum():,.0f}",
      "(should be about 140,733,247,081)")

prod = pd.read_csv(OUT / "week5_turnover_by_product.csv")[["product_id", "product_name", "cogs"]]
prod = prod.merge(prod_avg, on="product_id", how="left").merge(
    products[["product_id", "category"]], on="product_id", how="left")
prod["turnover"] = prod["cogs"] / prod["avg_inventory_value"]
prod["days_of_inventory"] = DAYS / prod["turnover"]
prod.sort_values("turnover").to_csv(OUT / "week5_turnover_by_product_corrected.csv", index=False)

cat = prod.groupby("category").agg(
    products=("product_id", "size"),
    cogs=("cogs", "sum"),
    avg_inventory=("avg_inventory_value", "sum"))
cat["turnover"] = cat["cogs"] / cat["avg_inventory"]
cat["days_of_inventory"] = DAYS / cat["turnover"]
cat.sort_values("turnover").to_csv(OUT / "week5_turnover_by_category_corrected.csv")
```

```
print(cat.sort_values("turnover").round(2).to_string())

import pandas as pd
import matplotlib.pyplot as plt
from pathlib import Path

OUT = Path("/content/week-05/output")
OUT.mkdir(parents=True, exist_ok=True)

br = pd.read_csv(OUT / "week5_turnover_by_branch.csv", index_col="branch_id")
aging = pd.read_csv(OUT / "week5_aging_buckets.csv", index_col="age_bucket")
cat = pd.read_csv(OUT / "week5_turnover_by_category_corrected.csv", index_col="category")
prod = pd.read_csv(OUT / "week5_turnover_by_product_corrected.csv")
aging_prod = pd.read_csv(OUT / "week5_aging_by_product.csv")

overall_turnover = br["cogs"].sum() / br["avg_inventory"].sum()

# Chart 1: turnover by branch
br_sorted = br.sort_values("turnover")
fig, ax = plt.subplots(figsize=(8, 5))
ax.barh(br_sorted.index, br_sorted["turnover"], color="steelblue")
ax.axvline(overall_turnover, linestyle="--", color="grey", label=f"Overall {overall_turnover:.2f}x")
ax.set_xlabel("Turnover (COGS / average inventory)")
ax.set_title("Turnover by branch: similar stock, very different sales")
ax.legend()
fig.tight_layout()
fig.savefig(OUT / "week5_chart1_turnover_by_branch.png", dpi=150)
plt.show()

# Chart 2: stock value by age bucket
order = ["0-30", "31-90", "91-180", "181-365", ">365"]
ag = aging.reindex(order)
fig, ax = plt.subplots(figsize=(8, 5))
ax.bar(ag.index, ag["pct_of_value"], color="seagreen")
for i, v in enumerate(ag["pct_of_value"]):
    ax.text(i, v + 1, f"{v:.1f}%", ha="center")
ax.set_ylabel("% of stock value")
ax.set_xlabel("Age of stock on hand (days)")
ax.set_title("Stock on hand by age (approximate, see data issues)")
fig.tight_layout()
fig.savefig(OUT / "week5_chart2_aging.png", dpi=150)
plt.show()

# Chart 3: turnover by category
cat_sorted = cat.sort_values("turnover")
fig, ax = plt.subplots(figsize=(8, 6))
ax.barh(cat_sorted.index, cat_sorted["turnover"], color="darkorange")
ax.axvline(overall_turnover, linestyle="--", color="grey")
ax.set_xlabel("Turnover")
ax.set_title("Turnover by category")
fig.tight_layout()
fig.savefig(OUT / "week5_chart3_turnover_by_category.png", dpi=150)
plt.show()

# KPI summary
over_365 = ag.loc[">365", "pct_of_value"]
kpis = {
    "Overall turnover (x)": round(overall_turnover, 2),
    "Days of inventory (2,192-day period)": round(2192 / overall_turnover),
    "Average inventory value (month-end)": round(br["avg_inventory"].sum(), 0),
    "Total COGS": round(br["cogs"].sum(), 0),
    "Lowest-turnover branch": f"{br_sorted.index[0]} ({br_sorted['turnover'].iloc[0]:.2f}x)",
    "Highest-turnover branch": f"{br_sorted.index[-1]} ({br_sorted['turnover'].iloc[-1]:.2f}x)",
    "Stock value older than 365 days (%, approx.)": over_365,
    "Products with average age over 365 days": int((aging_prod["avg_age_days"] > 365).sum()),
    "Total products": len(aging_prod),
}
kpi_df = pd.Series(kpis, name="value").to_frame()
kpi_df.to_csv(OUT / "week5_kpis.csv")
print(kpi_df.to_string())
```

```
print("\nSaved to", OUT)

import shutil
from google.colab import files

shutil.make_archive("/content/week05_outputs", "zip", "/content/week-05/output")
files.download("/content/week05_outputs.zip")
```